

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Equipping LLMs with the ability to actively search and integrate external knowledge is a general prerequisite for solving the complex, open-ended, and real-world tasks that increasingly define modern applications. Existing approaches, however, tend to fall into two camps: they either rely on prompting to elicit a model’s innate agentic behavior, or they hit performance ceilings and collapse once RL is applied to demanding interactive settings, leaving the broad agentic potential of these models largely untapped. To address this, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, a general-purpose RL algorithm designed for robust agent training through sequence-level optimization and dynamic sample filtering. We train our models purely through RL to interleave multi-turn search and reasoning, obviating the need for supervised demonstration data of any kind. Across a diverse suite of QA benchmarks, our 7B model improves over a comparable previous work by **34.1%**, and even outperforms the 14B model from previous work on demanding multi-hop QA such as HotpotQA by nearly **9% relative**, all while maintaining exceptional training stability. The same recipe transfers without modification to larger models and to distinct reasoning domains, pointing to a broadly applicable route for scaling capable agents.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) have demonstrated exceptional performance across a broad spectrum of tasks, from mathematical reasoning (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025) and coding (Zheng et al., 2023; Yang et al., 2024) to creative writing (Chakrabarty et al., 2024; Marco et al., 2024). Yet across all of these domains a fundamental limitation persists: their knowledge is inherently static, bounded by the data on which they were trained. A general and increasingly dominant strategy for overcoming this knowledge cutoff is to equip LLMs with search capabilities, turning them into agents that can actively query external and evolving knowledge sources (Jin et al., 2025b). Search is a canonical instance of the broader tool-calling paradigm (Schick et al., 2023), in which a model learns to interact with an external tool to solve problems it cannot answer on its own. Mastering any such skill requires learning a complex, multi-step policy, which naturally frames the task as a sequential decision-making problem well suited to Reinforcement Learning (RL).

Unlike Supervised Fine-Tuning (SFT), which relies on costly static demonstrations and fails to teach exploration, RL provides a general framework for LLMs to learn effective policies through trial-and-error (Chu et al., 2025). Consequently, value-free methods such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024) have become a widely adopted paradigm, prized for their simplicity and reduced memory overhead. However, despite their success on more constrained tasks, applying GRPO-style optimization to open-ended interactive settings—of which search is a representative example—exposes critical instabilities (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). This fragility stems from two fundamental and largely task-agnostic flaws. First, as identified by Zheng et al. (2025), GRPO’s token-level objective is ill-posed whenever it is paired with a sequence-level reward, creating high-variance gradients that destabilize training. Second, the sparse rewards inherent to search and to many other long-horizon reasoning tasks often yield sample groups with homogeneous outcomes (e.g., all successes or all failures), causing the advantage signal to collapse and providing no learning signal, which severely hinders sample efficiency (Yu et al., 2025; Liu et al., 2025).

To address these core challenges of instability and inefficient learning, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. Our algorithm synthesizes and refines key principles from recent policy optimization research into a general recipe for training reasoning agents. DSPO adopts the sequence-level optimization of GSPO (Zheng et al., 2025) to match the unit of optimization with the unit of reward. Because rewards in these settings are naturally assigned to entire trajectories, aligning the optimization objective with the reward signal fundamentally sta-

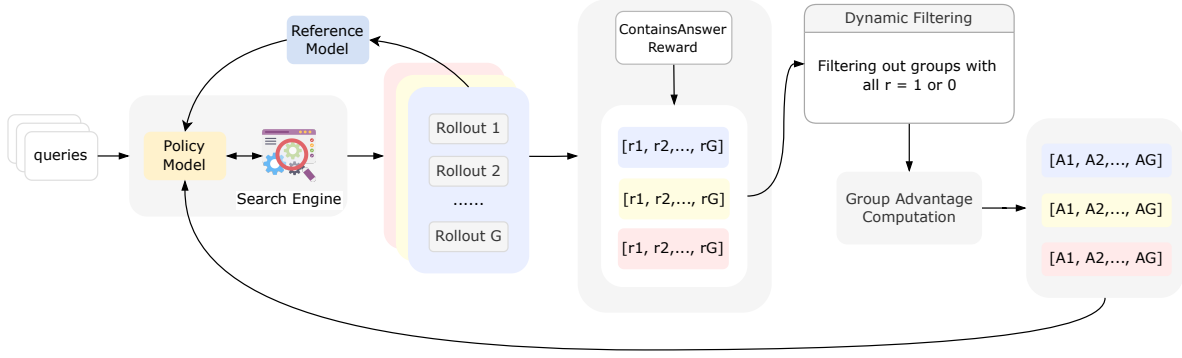


Figure 1: An overview of the general DSPO training loop. For a given query, the policy model generates a group of G trajectories by interacting with the environment. Each trajectory is assigned a sparse terminal reward. The **dynamic filter** discards groups with homogeneous outcomes and keeps sampling until a batch is filled, ensuring that every training batch provides an effective advantage signal. Advantages are then computed and used to update the policy model through a sequence-level objective. Although illustrated here with a search environment, the loop is agnostic to the underlying tool or task.

bilizes learning for essentially any long-horizon reasoning task. Furthermore, DSPO incorporates a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025), which actively constructs training batches from rollout groups containing both successful and unsuccessful outcomes for each prompt. This guarantees that the advantage signal $\hat{A}_i$ remains effective and stable regardless of how sparse the reward is. By integrating these two components into a single, coherent framework, DSPO provides a stable and high-performance algorithm that applies broadly to complex, multi-turn search and reasoning—and, as we show, transfers to further model scales and reasoning domains. Our model achieves a **34.1% relative improvement** over a leading 7B baseline (Jin et al., 2025b) and even surpasses its 14B counterpart (Jin et al., 2025a) on demanding multi-hop benchmarks such as HotpotQA, outperforming it by **nearly 9% relative** (0.613 vs. 0.563).

In summary, our main contributions are as follows:

- We propose DSPO, an improved and broadly applicable RL algorithm that overcomes the core instability and sample-inefficiency issues in training agentic reasoning models. It achieves this by unifying two general principles into a single cohesive framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for a dense and effective learning signal—principles that are not tied to any particular task or model family.
- We demonstrate DSPO’s substantial and consistent performance gains through rigorous benchmarking across a diverse set of QA tasks. Our 7B model achieves a **34.1% relative improvement** over a comparable 7B baseline and, more strikingly, outperforms its 14B counterpart on demanding multi-hop QA, achieving a **nearly 9% relative gain** on HotpotQA (0.613 vs. 0.563). We further show that these gains carry over to larger models and to a distinct reasoning domain.
- We provide extensive empirical evidence for DSPO’s superior training stability, showing that it enables a reliably stable learning trajectory across settings. Crucially, all results are obtained using only a basic BM25 retriever, isolating the performance gains to the robustness of the algorithm itself rather than to retrieval engineering.

2 RELATED WORK

2.1 RL FOR LLMs

The landscape of RL for LLMs has evolved rapidly, moving from foundational Reinforcement Learning from Human Feedback (RLHF) methods built on PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) toward simpler, direct-optimization frameworks such as DPO (Rafailov et al., 2023). A key shift toward value-free optimization is marked by Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which simplifies training by deriving a reward signal directly from group statistics and has been applied across a wide

range of reasoning tasks. However, GRPO’s token-level objective is known to cause training instability (Liu et al., 2025; Cui et al., 2025), prompting several targeted improvements. GSPO addresses this by shifting to a sequence-level objective to match the unit of reward (Zheng et al., 2025), while DAPO tackles inefficient learning from sparse rewards through a dynamic outcome-based sampling mechanism (Yu et al., 2025). In a similar vein, GMPO stabilizes the token-level objective using a geometric-mean aggregation to reduce sensitivity to outliers (Zhao et al., 2025). Despite these advances, we observe that these algorithms still encounter training collapse or performance bottlenecks in demanding interactive settings. Building on this body of work, we propose an improved algorithm that synthesizes the complementary principles of sequence-level optimization and dynamic filtering into a single unified method, designed to be robust not only for autonomous search agents but for long-horizon reasoning tasks more broadly.

2.2 LLMs WITH AGENTIC RETRIEVAL

To mitigate the static-knowledge limitations of LLMs across tasks, RAG integrates external retrievers that dynamically incorporate evolving information (Lewis et al., 2020; Gao et al., 2023). Classic RAG frameworks employ dense retrievers to fetch relevant documents, which are then concatenated into the LLM’s input for generation (Karpukhin et al., 2020). Such approaches, however, often rely on fixed pipelines that limit autonomy in complex, multi-turn scenarios. More recently, research has moved toward agentic paradigms in which LLMs act as autonomous agents capable of planning, searching, and reasoning iteratively. Frameworks like ReAct synergize reasoning and acting, enabling LLMs to interact with tools for tasks such as web navigation (Yao et al., 2023), while multi-agent systems such as AutoGen facilitate collaborative workflows (Wu et al., 2024). A growing line of work emphasizes agentic RAG and RL integration, where agents enhance retrieval through decision-making. Wu et al. (2025) introduce Agentic Reasoning, a framework that integrates external tools for streamlined LLM reasoning, and several RL-integrated approaches (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) train LLMs to interleave reasoning and search using RL alone. This end-to-end paradigm internalizes agentic capabilities and avoids the engineering overhead of multi-agent frameworks, but existing methods still grapple with training instability and performance bottlenecks whenever they are pushed into open-ended domains. Our work directly confronts these bottlenecks: DSPO offers a robust and efficient training framework that ensures stable policy optimization, enabling LLMs to learn effective multi-turn strategies that generalize across benchmarks, model scales, and reasoning domains.

3 METHODOLOGY

In this section, we first formulate agentic search as an RL problem—used here as a concrete and representative instance of long-horizon, sparse-reward reasoning—and review prior policy optimization algorithms, highlighting their limitations in this and similar settings. We then introduce our proposed algorithm, **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, detailing its core components for training stability and training efficiency. Finally, we present the integrated training algorithm. Throughout, we keep the formulation deliberately general so that the same components apply beyond the search setting used for exposition.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. Training LLMs via RL commonly relies on policy gradient methods such as PPO (Schulman et al., 2017), a widely used algorithm for LLM alignment. It optimizes a policy π_θ by maximizing a clipped surrogate objective using samples drawn from an old policy $\pi_{\theta_{\text{old}}}$. The objective, averaged over tokens, is given by:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. However, PPO relies on a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) (Schulman et al., 2015), which introduces significant memory overhead and can itself become a source of instability.

To address this, GRPO (Shao et al., 2024) was proposed as a lighter-weight alternative. GRPO eliminates the need for a value model by sampling a group of G responses $\{y_i\}_{i=1}^G$ for a given prompt x and computing each response’s advantage by normalizing its reward against the group’s statistics. Like PPO, it optimizes the objective at the token level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

Crucially, all tokens within a given response y_i share the same advantage $\hat{A}_i$, which is derived from the sequence-level reward—a mismatch that becomes increasingly problematic as trajectories grow longer.

Agentic Search as a Markov Decision Process. We model the iterative process of agentic search and reasoning as a sequential decision-making problem, formalized as a discrete-time, finite-horizon Markov Decision Process (MDP). We use search as a concrete instantiation, but the formulation applies to any tool-augmented reasoning task with the same structure.

- **State (s_t):** At turn t , the state s_t encodes the entire interaction history: the initial question q , all previously generated thoughts and queries, and the evidence returned by the environment. Because such results must be interpreted and integrated into subsequent decisions, the state grows with the trajectory, creating the kind of long-horizon dependencies that conventional token-level RL struggles to optimize in any tool-use setting.
- **Action (a_t):** An action is a full textual segment generated by the policy π_{θ} , consisting of free-form reasoning followed by a decision. The action terminates either with a `</tool_call>` token, which triggers a tool call (here, a search) and places the returned content within `</tool_response>`, or with a `</answer>` token, which ends the trajectory. This structured action space forces the agent to learn not only *what* to generate but also *when* to act—an aspect that introduces substantial variability in trajectory length across tasks.
- **Policy (π_{θ}):** The policy π_{θ} is the underlying LLM, generating tokens autoregressively conditioned on the state. It conditions on the full history, yet the optimization target is computed only over the model-generated thoughts and actions, masking out content returned by the environment (Jin et al., 2025b). Optimizing such a policy requires credit assignment over long sequences in which many intermediate reasoning steps receive no direct supervision or reward—a challenge common to long-horizon reasoning and one that further motivates stable sequence-level optimization.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records all reasoning and tool interactions taken for a given problem, including both model-generated actions and environment-returned content. Because the final reward is assigned to the trajectory as a whole, the optimization problem is fundamentally sequence-level regardless of the specific task: every early decision can influence the eventual correctness of the answer.
- **Reward ($R(\tau)$):** We adopt a deliberately simple, sparse terminal reward that requires no reward model: a trajectory receives $R = 1$ if the final answer contains the ground-truth text, and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we explicitly model the search engine, denoted $\mathcal{S}$, as part of the environment; more generally, $\mathcal{S}$ can be read as any external tool the policy queries. The policy LLM π_{θ} learns to generate trajectories by interleaving reasoning with calls to $\mathcal{S}$. The overall optimization problem is to find a policy that maximizes the expected reward, regularized by a KL divergence term to prevent large deviations from a reference policy π_{ref} :

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x;\mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_{\theta}(y|x;\mathcal{S}) || \pi_{\text{ref}}(y|x;\mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_{\theta}(\cdot|x;\mathcal{S})$ signifies that the trajectory y is generated through a multi-step process involving both the policy’s token generation and the information returned by the environment $\mathcal{S}$.

Motivation for DSPO. While frameworks like Search-R1 (Jin et al., 2025b) have successfully framed agentic search as an RL problem, applying conventional algorithms such as PPO or GRPO faces significant hurdles that are common to open-ended interactive tasks. The open-ended nature of these environments exacerbates the instability of token-level optimization. A core issue is the fundamental mismatch between the unit of sequence-level reward assignment and the unit of token-level optimization (Zheng et al., 2025). This discrepancy leads to high-variance gradient estimates that accumulate over long trajectories, often culminating in policy collapse. Furthermore, the sparse binary reward signal means many training batches contain only successful or only unsuccessful trajectories, yielding a degenerate advantage and thus no learning signal, which drastically reduces sample efficiency (Yu et al., 2025; Liu et al., 2025). Because both failure modes arise from the structure of long-horizon, sparse-reward learning rather than from search per se, an algorithm that counteracts them should benefit a broad range of tasks; DSPO is designed to do exactly that.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO introduces two key innovations over prior methods: (1) it performs policy optimization at the sequence level, aligning the training objective with the trajectory-based reward structure, and (2) it incorporates a dynamic filtering mechanism that ensures every training batch provides a high-quality, non-zero learning signal. Neither component is specialized to search, which is what allows the same procedure to carry over to other models and reasoning domains. The entire training process, integrating both components, is depicted in Figure 1.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Inspired by GSPO (Zheng et al., 2025), we replace the unstable token-level importance ratio with a theoretically grounded sequence-level counterpart that is well defined for trajectories of any length. The sequence-level importance ratio $s_i(\theta)$ for a response y_i is defined as the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization is crucial for reducing variance and for keeping $s_i(\theta)$ within a consistent numerical range regardless of sequence length—a property that is vital for stable clipping and that holds uniformly across tasks with widely varying trajectory lengths.

Gradient Analysis. The following analysis makes clear why DSPO improves stability in a task-independent way. The gradient of the token-level GRPO objective (unclipped) scales each token’s log-probability gradient by a noisy, token-specific weight $r_{i,t}(\theta)$. In contrast, the gradient of our sequence-level objective scales the average log-probability gradient of the entire sequence by a single, more stable sequence-level weight $s_i(\theta)$:

$$\nabla_\theta J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_\theta J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (8)$$

By applying a single, holistic correction factor to the entire trajectory, DSPO avoids the accumulation of token-level noise that plagues prior methods, yielding fundamentally more stable training for long sequences of any kind. In parallel, the dynamic filtering mechanism guarantees a well-behaved advantage signal $\hat{A}_i$ by constructing training batches from rollout groups that contain both successes and failures, thereby preventing wasted samples in any sparse-reward environment.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The sparse binary nature of our reward function poses a challenge for group-based advantage estimation—one shared by essentially any outcome-only reward. If all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined, so such batches provide no useful gradient and waste computation.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```

To overcome this, DSPO incorporates a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, we retain only groups of trajectories that contain a mix of successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This ensures that the reward variance within every training group is non-zero, guaranteeing a meaningful advantage signal. This dynamic selection curates a high-quality batch for each policy update, effectively converting a sparse-reward problem into a dense and efficient learning signal—a transformation that applies whenever rewards are sparse, not just in the search setting.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

By integrating these components, we arrive at the final DSPO objective. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, we compute the advantage $\hat{A}_i$ using group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_{θ} is updated by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. Crucially, during likelihood calculation we apply loss masking to all tokens returned by the tool, following Jin et al. (2025b). This ensures the model learns to *use* external knowledge for reasoning rather than simply to reproduce it, and the same masking principle extends to any tool whose outputs should not be treated as policy actions. The full training process is detailed in Algorithm 1.

4 EXPERIMENTS

In this section, we conduct a series of experiments to empirically validate the effectiveness and breadth of our proposed Dynamic-filter Sequence-level Policy Optimization (DSPO) algorithm. Our primary objectives are to demonstrate that

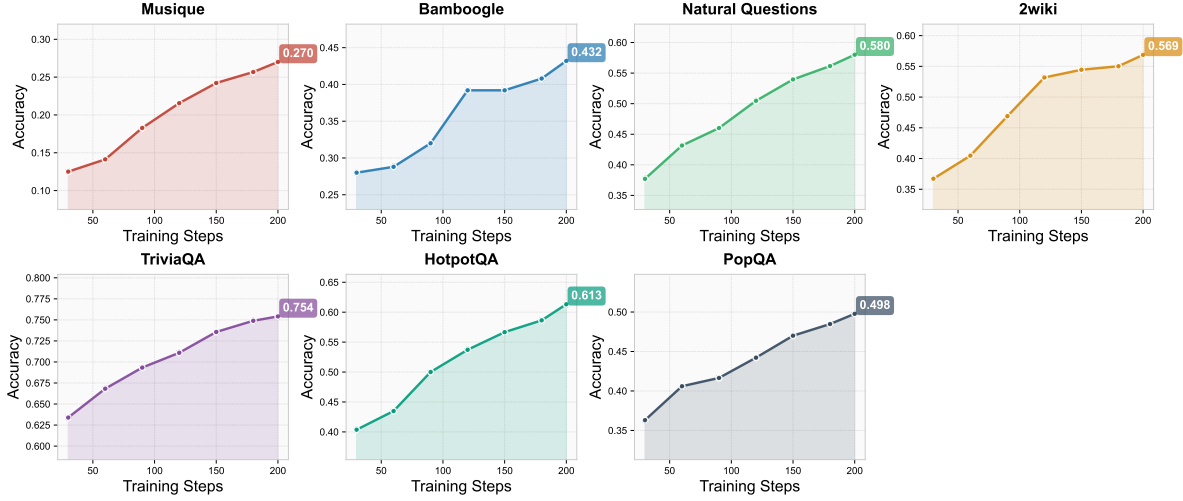


Figure 2: **Validation performance of DSPO across seven diverse QA benchmarks during training.** The steady, monotonic increase in accuracy across every benchmark confirms that DSPO’s reward improvement translates directly into broad generalization, indicating that the method learns a robust and transferable search-and-reasoning policy rather than overfitting to any single dataset.

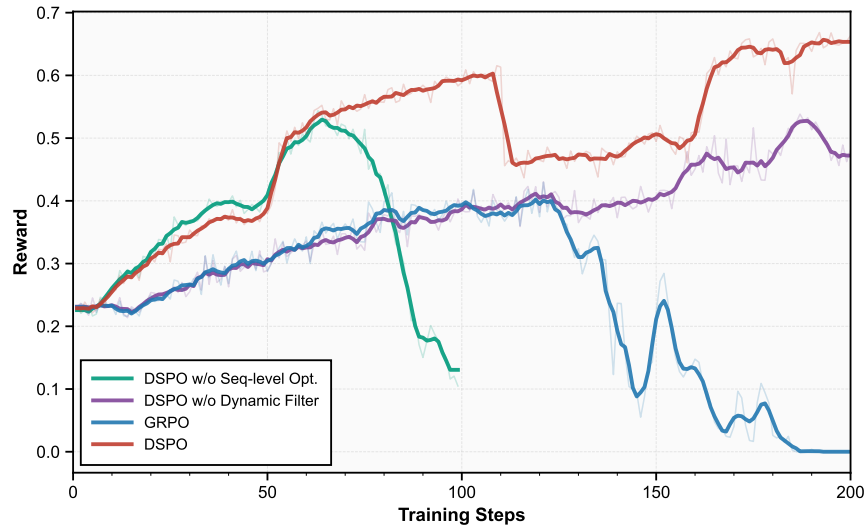


Figure 3: **Training reward dynamics of DSPO and its ablations.** A comparative view of the learning curves. DSPO (red) exhibits stable, monotonic improvement. In contrast, token-level variants (green, blue) suffer catastrophic policy collapse, while the sequence-level variant without our filter (purple) plateaus at a suboptimal level. This pattern illustrates a general trade-off between the two components rather than a search-specific artifact.

DSPO: (1) achieves strong and consistent performance across a diverse suite of challenging question-answering benchmarks; (2) exhibits significantly enhanced training stability, avoiding the catastrophic collapse that plagues baseline methods; and (3) derives its performance gains from the synergistic combination of its core components. We further probe how far these benefits extend by scaling to larger models and by transferring the method to a distinct reasoning domain.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), and as shown in Table 1, we use a single general-purpose prompt template to instruct the model’s actions during the search task, comprising `<think>`, `<tool_call>` and `<answer>` segments.

The template is intentionally minimal and task-agnostic, encoding only the reason–act–answer structure rather than any dataset-specific instructions.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: The single, task-agnostic prompt template used across all of our search experiments.

Benchmarks and Baselines. To provide a rigorous evaluation, our experimental design adheres to the established protocol of Search-R1 (Jin et al., 2025b). We train on a composite dataset containing the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018), and then assess generalization on the test sets of seven diverse QA benchmarks that together span single-hop, multi-hop, and long-tail entity questions: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022). Notably, five of these benchmarks are never seen during training, so strong test performance directly reflects out-of-distribution generalization rather than in-domain fitting.

Our comparison suite pairs strong external baselines with critical internal ablations. The external baselines are the Qwen2.5-7B and 14B models trained with PPO and GRPO from the Search-R1 framework (Jin et al., 2025b;a). To deconstruct our method, we include two internal ablations: (1) **DSPO w/o dynamic filter**, which is equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reverts to a strong token-level policy, DAPO (Yu et al., 2025).

For implementation, we use the Qwen2.5-7B-Instruct model as the starting checkpoint for our main experiments. Our experiments build on the VeRL framework (Sheng et al., 2025), for which we adapted the provided Search-R1-like example code and scripts to suit our methodology. We benchmark DSPO against a comprehensive suite of baselines; for external comparison, we use the PPO and GRPO methods from the Search-R1 framework (Jin et al., 2025b;a). Crucially, because our work uses a modified reward function, we retrained these models under identical experimental conditions to ensure a fair comparison, and the results of these retrained models serve as our primary external benchmarks.

Implementation and Evaluation. To isolate the benefits of our algorithm from those of retrieval engineering, all RL experiments deliberately employ a standard, unmodified BM25 retriever. This deliberately conservative setup ensures that observed improvements are attributable to the learned policy rather than to a stronger retriever, and it establishes a lower bound: gains realized here should only widen with more capable retrievers. Across all methods, models are trained with a sparse, binary reward based on substring Exact Match (subEM) of the final answer, and subEM also serves as the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

To provide a holistic view of our algorithm’s effectiveness across the full benchmark suite, we present a comprehensive comparison in Table 2. Given the synergistic nature of DSPO’s components, we find it most illustrative to report the main results alongside the ablation study. This single table juxtaposes DSPO against both external state-of-the-art baselines and its own ablated variants, offering a clear and direct assessment of its broad superiority and the indispensability of each core component.

Comparison with Baselines. The results in Table 2 underscore DSPO’s broad superiority across the benchmark suite. Our DSPO-trained 7B agent attains an average score of 0.531, establishing a new state of the art and improving on the same-sized Search-R1 (GRPO, 7B) model by 34.1% relative. More strikingly, our 7B agent edges out the much larger Search-R1 14B models under both GRPO and PPO on average. Because these gains hold consistently across single-hop, multi-hop, and long-tail benchmarks alike, they provide strong evidence that the improvement stems from a more effective and stable learning algorithm rather than from an over-reliance on model scale.

Analysis of Ablations. The ablation results, also reported in Table 2, demonstrate that both of DSPO’s components are indispensable across the board. First, removing the dynamic filter (‘w/o dynamic filter’) causes a catastrophic drop

Table 2: Comprehensive comparison of DSPO with baselines and ablation variants across all seven QA benchmarks. Baselines include Search-R1 models (7B & 14B) trained with GRPO and PPO (Jin et al., 2025b;a). Ablations remove key components: ‘w/o dynamic filter’ and ‘w/o seq-level opt.’. Original EM scores from Search-R1 are shown in parentheses; to keep the evaluation consistent, we retrained and re-evaluated these models under our adjusted rewards. Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

in performance, with the average score falling to 0.313. This highlights its critical role: without the filter, the sequence-level objective is starved of a useful learning signal by homogeneous-reward batches. Second, ablating sequence-level optimization (‘w/o sequence-level opt.’) also leads to substantial degradation, yielding an average score of 0.406. While this token-level variant outperforms the filter-less one, it falls well short of the full DSPO model, and, as we show in the next section, it remains prone to catastrophic training instability. Together these results confirm that the synergy is essential and general: sequence-level updates provide stability, while the dynamic filter transforms sparse rewards into an efficient learning signal, and neither alone suffices.

Beyond aggregate metrics, we observe that DSPO gives rise to sophisticated search behaviors, including recognizing irrelevant results, reformulating queries, and verifying answers over multiple turns (see Appendix A for detailed trajectory examples). All of these behaviors emerge from pure RL training with DSPO, without any behavioral demonstrations, suggesting that such strategies are learned rather than hand-specified and could transfer to other tool-use settings.

4.3 ANALYSIS OF TRAINING DYNAMICS

To empirically validate our claims regarding stability and efficiency, we analyze the training reward dynamics of DSPO, its ablations, and key baselines. Figure 3 offers a compelling visualization of these dynamics that reinforces our core algorithmic choices and points to their general applicability.

DSPO (red) exhibits a smooth, monotonic ascent, efficiently converging to the highest reward level and empirically confirming the stability afforded by its sequence-level objective. In stark contrast, the **token-level methods**—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—suffer catastrophic policy collapse early in training: their rewards plummet after a brief initial rise, a clear manifestation of the instability caused by high-variance, token-level gradient updates. Meanwhile, **DSPO w/o Dynamic Filter** (purple), which uses sequence-level updates but lacks an efficient learning signal, remains stable yet plateaus at a markedly suboptimal ceiling. These qualitatively distinct behaviors reveal that DSPO’s synergy of sequence-level stability and dynamic filtering is what enables robust and effective policy optimization—a mechanism that is not specific to any one benchmark.

To ensure that improvements in training reward reflect genuine generalization rather than reward hacking, we track validation performance on held-out benchmarks throughout training. As illustrated in Figure 2, DSPO’s validation accuracy on NQ, HotpotQA, and the other diverse benchmarks rises consistently, mirroring its stable reward curve. This tight correlation across datasets confirms that the agent is learning a genuinely generalizable search-and-reasoning policy rather than exploiting artifacts of the training distribution.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

To further probe the breadth of our approach, we extend the evaluation along two axes that test how far DSPO’s benefits reach: model scale and reasoning domain.

Scalability to Larger Models. We first ask whether the stability benefits of DSPO carry over to larger parameter scales by training Qwen2.5-14B-Instruct with the same recipe. As detailed in Table 3, DSPO scales gracefully. The DSPO-trained 14B model attains an average accuracy of 60.6%, outperforming the strong GRPO-14B baseline (53.0%) by a relative margin of **14.3%**, with gains on every individual benchmark. These results indicate that the method leverages increased model capacity effectively rather than saturating, and that its advantage over standard baselines persists—and even grows—as models get larger.

Table 3: Scalability analysis on Qwen2.5-14B-Instruct across all seven QA benchmarks. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. To test whether DSPO’s benefits extend beyond agentic search, we apply it unchanged to single-turn mathematical reasoning using the Qwen2.5 and Qwen3 model families. Table 4 reports the comparison on Math500 and Olympiad-Bench. DSPO consistently surpasses GRPO across both the 7B and 4B model sizes and across both benchmarks. Although this domain differs markedly from multi-turn search—it is single-turn and does not involve external tools—the same algorithm still improves over the baseline, providing direct evidence that DSPO’s core principles transfer to general reasoning domains rather than being tailored to search.

Table 4: Generalization to a distinct, single-turn mathematical reasoning domain. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

In this work, we tackled the critical instability and sample-inefficiency issues that plague RL for autonomous LLM agents. We introduced Dynamic-filter Sequence-level Policy Optimization (DSPO), an improved and general-purpose algorithm that ensures robust training through two complementary components: sequence-level optimization to prevent catastrophic policy collapse, and a dynamic outcome-based filter to turn sparse rewards into a consistently effective learning signal. Our experiments demonstrated that DSPO not only delivers strong performance across a diverse suite of challenging question-answering benchmarks but also exhibits superior training stability compared to prior methods, and that these benefits extend to larger models and to a distinct reasoning domain.

By enabling robust training from environmental feedback alone, DSPO offers a practical and efficient blueprint for building capable LLM agents without costly expert demonstrations. With this stable foundation, future work can integrate more advanced retrievers or extend DSPO to richer multi-tool settings. Because the challenges of sparse rewards and unstable policy gradients are intrinsic to long-horizon RL rather than specific to search, and because our mathematical-reasoning results already show the method transferring across domains, we expect DSPO’s principles to yield comparable gains in further domains such as code generation—a promising direction for broader validation. Ultimately, we believe the core tenets of DSPO—matching the optimization unit to the reward signal and guaranteeing signal density—are general design principles that will help drive the next generation of autonomous, tool-using AI.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

To provide insight into the learned search strategies, Table A.1 presents representative trajectories drawn from our model. These examples illustrate a range of general-purpose behaviors: (1) reformulating queries when initial searches return irrelevant results, (2) verifying information through follow-up searches, and (3) extracting comprehensive answers from retrieved evidence. Such behaviors are not tied to any single question type and are the kind of adaptive strategies one would hope to see transfer to other tool-use tasks.

Table A.1: Representative trajectories illustrating the model’s learned search and reasoning capabilities across different question types. The model exhibits sophisticated, broadly transferable behaviors including (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let's try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician